

Aplicație de Investiții - Bordean David Ioan

de Bordean David Ioan

Proiectul meu este o aplicație de consola realizată în C++, care simulează o platformă simplă de investiții. Ideea principală este ca un utilizator poate să își creeze un cont de investitor, să vadă ce instrumente financiare există pe piață, să cumpere sau să vândă active și să își urmărească portofoliul. Aplicația include mai multe tipuri de instrumente financiare, cum ar fi acțiuni, criptomonede și obligațiuni, ceea ce face proiectul mai interesant decât o simplă gestiune de obiecte.

Din punctul meu de vedere, proiectul este bine ales pentru a demonstra concepte importante de programare orientată pe obiect. Nu este doar un program care afișează niște meniuri, ci are o structură clară, cu clase separate, mostenire, polimorfism, interfețe, excepții și chiar design pattern-uri. Acest lucru arată că proiectul a fost gândit ca o aplicație mai mare, nu doar ca un exercițiu simplu.

Scopul proiectului

Scopul proiectului este de a crea o aplicație prin care se poate gestiona un portofoliu de investiții. Utilizatorul poate porni cu un anumit sold inițial, apoi poate cumpara instrumente disponibile în piață. După cumpărare, acestea apar în portofoliu, iar aplicația calculează automat valoarea totală, soldul liber și profitul.

Un alt scop important este simularea modificării prețurilor. În meniul principal există opțiunea de a trece o zi, iar prețurile se actualizează automat. Acest lucru face ca aplicația să para mai realistă, deoarece prețurile activelor financiare nu rămân fixe.

Structura generală a codului

Codul este împărțit în mai multe fișiere, fiecare clasă având fișierul ei .h și, unde este cazul, fișierul .cpp. Această separare ajută mult la organizare și face codul mai ușor de citit. De exemplu, clasele Acțiune, Crypto, Obligațiune, Investitor și Piață sunt separate clar, fiecare având rolul ei.

Un punct foarte bun este faptul că proiectul folosește o clasă abstractă de bază, InstrumentFinanciar. Aceasta permite ca toate instrumentele financiare să fie tratate în mod comun, chiar dacă fiecare are detalii specifice. Astfel, portofoliul investitorului poate conține mai multe tipuri de instrumente prin pointeri la clasa de bază.

Clasa InstrumentFinanciar

InstrumentFinanciar este una dintre cele mai importante clase din proiect, deoarece este clasa de bază pentru toate tipurile de active. Ea conține câmpuri comune, cum ar fi numele instrumentului, prețul curent, prețul de cumpărare și cantitatea.

Ce face bine această clasă este că obligă clasele derivate să implementeze metode precum calculeazăProfit, valoare și clone. Prin aceste metode virtuale, proiectul folosește

polimorfismul într-un mod corect. De asemenea, clasa validează prețurile și cantitatea, ceea ce ajută la evitarea unor date gresite în program.

Metoda afisează este utilă deoarece oferă o afișare comună pentru toate instrumentele, iar detaliile specifice sunt completate prin afisareExtra. Acest lucru reduce repetiția de cod.

Clasele Actiune, Crypto și Obligatiune

Clasa Actiune reprezintă acțiunile simple. Ea calculează profitul prin diferența dintre prețul curent și prețul de cumpărare, înmulțită cu cantitatea. Este o implementare clară și ușor de înțeles. În plus, clasa are și metoda getCatalog, care citește acțiunile din fișierul actiuni.in.

Clasa Crypto este asemănătoare cu Actiune, dar are în plus simbolul criptomonedei. Acest lucru este normal, deoarece criptomonedele sunt de obicei recunoscute prin simboluri precum BTC sau ETH. Și aici profitul este calculat simplu, iar datele pot fi încărcate din crypto.in.

Clasa Obligatiune este puțin mai complexă, deoarece include rata dobânzii și anii până la scadență. Aici profitul nu este calculat doar prin diferența de preț, ci include și dobânda. Acest lucru este un plus, deoarece arată că fiecare tip de instrument poate avea o logică proprie. Din punctul meu de vedere, această clasă arată cel mai bine avantajul mostenirii și al polimorfismului.

Clasa Investitor

Investitor este clasa care gestionează utilizatorul și portofoliul său. Ea conține numele investitorului, soldul liber, lista de instrumente deținute și istoricul tranzacțiilor. Această clasă este foarte importantă deoarece aici se află logica principală pentru cumpărare, vânzare, adăugare de fonduri și retragere de fonduri.

Un lucru foarte bun este că portofoliul este gestionat prin pointeri la InstrumentFinanciar, iar obiectele sunt copiate prin metoda clone. Asta înseamnă că se păstrează tipul real al obiectului, chiar dacă este stocat prin pointer la clasa de bază. De asemenea, clasa are destructor, constructor de copiere și operator de atribuire, ceea ce arată atenție la gestionarea memoriei.

Metodele cumpara și vinde sunt bine făcute. La cumpărare se verifică dacă există fonduri suficiente, iar dacă instrumentul există deja în portofoliu, se actualizează cantitatea și prețul mediu de cumpărare. La vânzare se verifică dacă există cantitate suficientă și, dacă ajunge la zero, instrumentul este eliminat automat din portofoliu.

Clasa Piata

Piata reprezintă piața financiară a aplicației. Ea încarcă acțiunile, criptomonedele și obligațiunile din fișiere și permite afișarea sau căutarea lor. Eu cred că este o clasă bine gândită, deoarece separă clar piața de investitor. Investitorul nu trebuie să știe cum sunt stocate toate instrumentele, ci doar folosește piața pentru a cumpara sau a actualiza prețurile.

Un aspect foarte bun este folosirea pattern-ului Singleton. În aplicație are sens să existe o singură piață principală, iar Piata::getInstance() rezolvă această problemă. De asemenea, piața folosește Observer pentru a anunța investitorii când prețurile se modifică și Strategy pentru modul în care se generează modificările de preț. Metoda actualizeazaPrețuri este una dintre cele mai importante, deoarece simulează trecerea unei zile.

Interfetele si clasele auxiliare

IObserverPiata este o interfata simpla, dar utila. Ea permite ca un investitor sa fie notificat atunci cand piata se modifica. Acest lucru face codul mai flexibil, pentru ca in viitor ar putea exista si alte clase care sa observe piata.

IStrategiePret si StrategieAleatoare sunt folosite pentru pattern-ul Strategy. Momentan exista o strategie aleatoare, care modifica preturile între 90% și 110% din valoarea initiala. Este o idee buna, deoarece în viitor se pot adăuga ușor strategii mai complexe.

Tranzactie<T> este o clasa template folosită pentru istoricul tranzacțiilor. Ea reține numele instrumentului, tipul tranzactiei, cantitatea și prețul. Folosirea unui template este un plus, deoarece arata ca proiectul foloseste și concepte mai avansate din C++.

Clasele din Exceptii.h sunt foarte utile pentru tratarea erorilor. În loc sa se foloseasca doar mesaje simple sau coduri de eroare, proiectul are excepții clare pentru fonduri insuficiente, cantitate insuficienta, instrument negasit si valori invalide.

Ce face bine proiectul

Cel mai mare avantaj al proiectului este organizarea. Codul este impartit logic, iar clasele au responsabilitati clare. De asemenea, folosirea mostenirii si a polimorfismului este potrivita pentru tema aleasa. Instrumentele financiare chiar au nevoie de comportamente comune si comportamente specifice.

Un alt lucru foarte bun este faptul ca proiectul nu se limiteaza la clase simple, ci include si design pattern-uri. Singleton, Observer si Strategy sunt folosite in contexte potrivite, nu doar puse artificial.

Meniul din main.cpp este destul de complet și permite folosirea aplicatiei fara sa fie nevoie de modificari in cod. Utilizatorul poate crea conturi, se poate conecta, poate vedea piata, poate cumpara si vinde. Pentru un proiect de student, functionalitatea este destul de bogata.

Probleme si posibile imbunatatiri

O imbunatatire ar fi salvarea conturilor in fisiere. Momentan investitorii exista doar cat timp ruleaza programul. Ar fi util ca portofoliile și istoricul tranzactiilor sa fie salvate si reincarcate la pornire.

De asemenea, unele validari ar putea fi extinse. De exemplu, numele instrumentelor nu pot avea spatii, deoarece citirea din fisiere se face cu >>.

Functii care nu sunt folosite momentan

Exista cateva functii care nu sunt folosite direct in meniul actual, dar care sunt utile pentru dezvoltari viitoare. De exemplu, getNrActiuniCreate si getNrCryptoCreate ar putea fi folosite pentru o pagina de statistici, unde se afiseaza cate obiecte de un anumit tip au fost create.

Metoda setStrategie din Piata nu este folosita în meniul actual, dar este foarte utila. In viitor, utilizatorul ar putea alege între mai multe strategii de modificare a preturilor, cum ar fi strategie conservatoare, agresiva sau bazata pe trend.

Metoda dezregistreaza ar putea fi folosita cand un investitor este sters sau cand se inchide un cont. De asemenea, areInstrument si getCantitateInstrument pot fi folosite pentru

afisari mai rapide sau pentru validari inainte de vanzare. setNume și setSoldLiber pot fi folosite într-un meniu de editare profil sau administrare cont.

Despre README

README-ul este util deoarece explica pe scurt ce face proiectul, ce clase exista și cum se compileaza aplicatia. Este important pentru ca o persoana care vede proiectul prima data poate intelege rapid structura lui și modul de rulare.

Concluzie

În concluzie, proiectul este bine realizat și arată o înțelegere a programării orientate pe obiect în C++. Nu este perfect și mai poate fi îmbunătățit, mai ales la partea de salvare a datelor și folosirea smart pointerilor, dar pentru un prim proiect este clar, functional si bine structurat. Cel mai important este că ideea proiectului este aplicată coerent, iar clasele chiar colaborează între ele într-un mod logic.